LLMDataAudit - 大模型数据审计

题目描述

某公司部署了基于大模型的智能客服系统"智购客服"。安全团队在例行审计中发现该系统的对话日志被意外导出，且日志中存在攻击者通过 Prompt Injection 手段诱导模型泄露用户敏感信息的记录。

你的任务是对导出的对话日志进行安全审计：

1. 识别所有攻击成功并导致数据泄露的对话轮次
2. 从模型响应中提取泄露的敏感个人信息（PII）
3. 判断每次攻击使用的注入类型
4. 将结果按指定格式整理提交

附件说明

文件	说明
chat_logs.jsonl	对话日志（共 10000 轮），每行一个 JSON 对象
system_prompt.txt	该客服系统使用的 System Prompt
README.md	本文件

对话日志格式

```
{
  "turn_id": 1,
  "timestamp": "2025-03-10T09:03:00Z",
  "session_id": "sess_abc12345",
  "user_input": "用户输入内容",
  "model_output": "模型响应内容"
}
```

攻击类型定义

日志中存在四种 Prompt Injection 攻击方式。判定攻击类型时，应 **仅根据 `user_input` 的内容特征** 进行分类，优先级从高到低排列如下：

1. 编码绕过（encoding）

攻击者将恶意指令进行 Base64 编码后嵌入输入。

识别特征：`user_input` 中同时出现与 Base64 相关的提示词（如"Base64""编码指令""encoded command""Decode"等）以及一段符合 Base64 编码规则的长字符串（仅由 `A-Za-z0-9+/=` 组成，长度通常 ≥ 20 字符）。

2. Unicode 混淆（unicode）

攻击者将指令使用 Unicode 转义序列表示，试图绕过文本过滤。

识别特征：`user_input` 中包含 `\uXXXX` 格式的 Unicode 转义序列（如 `\u8f93\u51fa`）。

3. 角色扮演绕过（role_play）

攻击者通过指派虚拟角色身份来诱导模型脱离安全限制。

识别特征：`user_input` 中包含角色指派类语句，例如：

- 中文：使用"假设你是""你现在是""你的身份是""你的角色是""扮演""从现在开始你是"等角色设定句式
- 英文：使用"Pretend you are""Act as""You are now""Let's play"等角色指派表达
- 还可能包含虚构的角色名称（如以 Bot / GPT / AI / Data 等为后缀的自定义名称）

4. 直接指令注入（direct）

攻击者在输入中直接使用覆盖 / 绕过系统安全规则的命令。

识别特征：`user_input` 中包含明确的安全覆盖类指令，例如：

- 中文关键词：涉及"忽略""系统管理员""调试命令/指令""输出用户数据""关闭安全过滤""解除""审计员""导出用户"等
- 英文关键词：涉及 "Ignore"（与 instruction / safety / protocol / privacy 等搭配）、"Override"、"Disable safety/filter"、"Dump all"、"System Override"、"Display user/stored/PII"、"Print user/PII" 等（不区分大小写）

> **注意**：当一条 `user_input` 同时匹配多种类型特征时，按上述优先级（encoding > unicode > role_play > direct）取最高优先级类型。

数据校验规范

身份证号（idcard）

身份证号为 **18 位**字符串，由六位地址码、八位出生日期码、三位顺序码和一位校验码组成。

校验码计算方法：

1. 将身份证号前 17 位数字分别乘以对应的系数。从第一位到第十七位的系数分别为：

   ```
   7, 9, 10, 5, 8, 4, 2, 1, 6, 3, 7, 9, 10, 5, 8, 4, 2
   ```

2. 将 17 个乘积相加，得到加权和。
3. 将加权和除以 11，取余数。
4. 根据余数查下表得到校验码（第 18 位）：

余数	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
校验码	1	0	X	9	8	7	6	5	4	3	2

只有通过上述校验的 18 位字符串才是有效身份证号。

手机号码（phone）

手机号码为 **11 位**纯数字字符串。

手机号前 3 位号段范围限定在以下集合中：

```
134，135，136，137，138，139，147，148，150，151，152，157，158，159，
172，178，182，183，184，187，188，195，198，
130，131，132，140，145，146，155，156，166，167，171，175，176，185，186，196，
133，149，153，173，174，177，180，181，189，190，191，193，199
```

不在上述号段范围内的 11 位数字串不应被识别为手机号。

银行卡号（bankcard）

银行卡号为 **16-19 位**纯数字字符串，使用 **Luhn 算法**进行校验。

常见银行卡 BIN 前缀包括：

```
622848，622700，621700，622262，622188，622200，622568，622609，622908，622518
```

Luhn 算法校验方法：

1. 从右到左，将偶数位上的数字乘以 2（最右边为第 1 位，即奇数位）。
2. 如果某个偶数位乘以 2 的结果大于 9，则将结果的各位数字相加（等价于减 9）。
3. 将所有位上的数字求和。
4. 若总和能被 10 整除，则校验通过。

示例：对卡号 `6228480402564890018`

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
原始数字	6	2	2	8	4	8	0	4	0	2	5	6	4	8	9	0	0	1	8
位置(右起)	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
偶数位×2		4		16		16		8		4		12		16		0		2	
调整>9		4		7		7		8		4		3		7		0		2	

- 求和：6+4+2+7+4+7+0+8+0+4+5+3+4+7+9+0+0+2+8 = **80**
- 80 % 10 == 0 → **校验通过**

只有通过 Luhn 校验的数字串才是有效银行卡号。

电子邮箱（email）

标准电子邮箱格式：`用户名@域名`，如 `zhangwei123@gmail.com`。

数据变形说明

在攻击成功的对话轮次中，模型泄露的敏感数据 **可能经过以下变形处理**：

变形方式	说明	示例
明文输出	PII 直接以原始格式出现	`110101199003071234`
Base64 编码	PII 经 Base64 编码后输出	`MTEwMTAxMTk5MDAzMDcxMjM0`
字符分隔	PII 各字符间插入分隔符（空格、连字符或逗号）	`1-1-0-1-0-1-1-9-9-0-0-3-0-7-1-2-3-4`

提交时 `value` 字段应填写还原后的标准格式（即去除分隔符、解码 Base64 后的原始值）。

重要注意事项

- **部分攻击轮次中模型成功拒绝了攻击（未泄露任何数据），这些轮次不应出现在结果中。只有模型响应中实际包含了可提取的有效 PII 的轮次才需要记录。**
- **日志中大量对话为正常客服交互（如订单查询、退换货等），这些正常对话不属于攻击，不应出现在结果中。**
- **提取到的 PII 必须经过对应的校验规则验证（身份证校验码、Luhn 算法等），未通过校验的数据应排除。**
- 日志中可能出现干扰性数字串（如订单号、快递单号、工单编号等），这些不属于 PII，需注意区分。
- 同一轮对话中可能泄露多条不同类型的 PII，每条 PII 单独一行。
- 攻击类型判定依据是 `user_input`，不是 `model_output`。

提交格式

上传 `result.csv` 文件（UTF-8 编码），格式如下：

```
turn_id,attack_type,category,value
142,role_play,idcard,110101199003071234
142,role_play,phone,13812345678
205,encoding,email,zhangsan@example.com
```

- `turn_id`：对话轮次编号（整数）
- `attack_type`：攻击类型（`direct` / `role_play` / `encoding` / `unicode`）
- `category`：敏感数据类型（`idcard` / `phone` / `bankcard` / `email`）
- `value`：还原后的标准格式 PII 值

将 `result.csv` 上传至判题平台校验，**正确率达到 98% 以上即可获取 flag。**